

UNIDAD 4: MACHINE LEARNIG

Contenido

CAPÍTULO 0.- MACHINE LEARNING. INTRODUCCIÓN	1
CAPÍTULO 1.- REGRESIÓN LOGÍSTICA	2
1.1. ¿Qué es la regresión logística?	2
1.2. ¿Por qué es importante la regresión logística?	3
1.3. ¿Cuáles son las aplicaciones de la regresión logística?	4
1.4. ¿Cómo funciona el análisis de regresión?	4
1.5. ¿Cómo funciona el modelo de regresión logística?	5
1.6. ¿Cuáles son los tipos de análisis de regresión logística?	7
1.7. ¿Cómo se compara la regresión logística con otras técnicas de ML?	8
BIBLIOGRAFÍA	10

CAPÍTULO 0.- MACHINE LEARNING. INTRODUCCIÓN

En los temas anteriores trabajamos con **regresión lineal simple** y **regresión lineal múltiple**, aprendiendo a ajustar modelos que explican **una variable continua** a partir de una o varias variables predictoras. Ese fue nuestro primer gran paso dentro del aprendizaje supervisado: entender cómo un modelo puede capturar relaciones lineales y utilizarlas para predecir.

Ahora vamos a **dar un paso más** y adentrarnos en técnicas que amplían ese marco y nos permiten abordar problemas distintos y, en muchos casos, más realistas. En concreto, introduciremos dos herramientas fundamentales en machine learning: la **regresión logística**, que nos permite modelar **variables categóricas** (especialmente problemas de clasificación binaria), y los **árboles de decisión**, modelos flexibles y muy interpretables capaces de capturar **relaciones no lineales y reglas de decisión complejas**.

Ambos métodos representan un avance natural respecto a la regresión lineal: mantienen la idea de aprender a partir de datos, pero amplían el tipo de problemas que podemos resolver y la forma en que los modelos se adaptan a la información disponible. A partir de aquí, empezaremos a ver cómo el aprendizaje automático nos ofrece un abanico mucho más amplio de posibilidades.



CAPÍTULO 1.- REGRESIÓN LOGÍSTICA

1.1. ¿Qué es la regresión logística?

Tal como vimos en los temas anteriores sobre **regresión lineal simple** y **regresión lineal múltiple**, donde modelamos relaciones entre variables continuas, ahora utilizamos el aprendizaje supervisado para abordar problemas de **clasificación**. A continuación, exploraremos el **modelo de regresión logística**, una técnica que permite predecir variables categóricas a partir de características numéricas.

La regresión logística es un algoritmo supervisado que se utiliza para resolver problemas de clasificación. A diferencia de la regresión lineal, cuya salida es continua, la regresión logística produce una salida **discreta**, como "Sí/No" o una clase entre varias posibles.

La **regresión logística** es una técnica de aprendizaje supervisado que permite **predecir variables categóricas a partir de características numéricas**. A diferencia de la regresión lineal, que modela relaciones entre variables continuas, la regresión logística se utiliza para resolver problemas de clasificación, produciendo resultados discretos como "Sí/No" o múltiples clases.

1.1.1. Ejemplos de aplicación:

- Clasificar correos como "Spam" o "No Spam".
- Diagnóstico médico: "Benigno" o "Maligno".
- Clasificación de artículos: "Deportes", "Política", "Ciencia", etc.
- Decisión crediticia: "Conceder crédito" o "No conceder crédito".

1.1.2. Casos prácticos

Caso práctico 1: Clasificación del Sistema Operativo

Utilizaremos un conjunto de datos con 170 registros para clasificar el sistema operativo de los usuarios que visitan un sitio web. Las clases posibles son:

- 0: Windows
- 1: Macintosh
- 2: Linux

Las características de entrada son:

- Duración de la visita (segundos)
- Páginas vistas
- Acciones realizadas (clicks, scroll, etc.)



- Valor de las acciones (ponderación de importancia)

Caso práctico 2: Predecir comportamiento visitante sitio Web

Por ejemplo, supongamos que desea adivinar si el visitante de su sitio web va a hacer clic en el botón de pago de su carrito de compras o no. El análisis de regresión logística analiza el comportamiento de los visitantes anteriores, como el tiempo que permanecen en el sitio web y la cantidad de artículos que hay en el carrito. Determina que, si anteriormente los visitantes pasaban más de cinco minutos en el sitio y agregaban más de tres artículos al carrito, hacían clic en el botón de pago. Con esta información, la función de regresión logística puede predecir el comportamiento de un nuevo visitante en el sitio web.

1.2. ¿Por qué es importante la regresión logística?

La regresión logística es una técnica importante en el campo de la [inteligencia artificial](#) y el [aprendizaje automático](#) (AI/ML). Los modelos de ML son programas de software que puede entrenar para realizar tareas complejas de procesamiento de datos sin intervención humana. Los modelos de ML creados mediante regresión logística ayudan a las organizaciones a obtener información procesable a partir de sus datos empresariales. Pueden usar esta información para el análisis predictivo a fin de reducir los costos operativos, aumentar la eficiencia y escalar más rápido. Por ejemplo, las empresas pueden descubrir patrones que mejoran la retención de los empleados o conducen a un diseño de productos más rentable.

A continuación, enumeramos algunos **beneficios del uso de la regresión logística en comparación con otras técnicas de ML**.

- **Simplicidad**

Los modelos de regresión logística son matemáticamente menos complejos que otros métodos de ML. Por lo tanto, puede implementarlos incluso si nadie de su equipo tiene una profunda experiencia en ML.

- **Velocidad**

Los modelos de regresión logística pueden procesar grandes volúmenes de datos a alta velocidad porque requieren menos capacidad computacional, como memoria y potencia de procesamiento. Esto los hace ideales para que las organizaciones que están empezando con proyectos de ML obtengan ganancias rápidas.

- **Flexibilidad**

Puede usar la regresión logística para encontrar respuestas a preguntas que tienen dos o más resultados finitos. También puede usarlo para preprocesar datos. Por ejemplo, puede ordenar los datos con un amplio rango de valores, como las transacciones bancarias, en un rango de valores más



pequeño y finito mediante la regresión logística. A continuación, puede procesar este conjunto de datos más pequeño mediante el uso de otras técnicas de ML para obtener un análisis más preciso.

- **Visibilidad**

El análisis de regresión logística ofrece a los desarrolladores una mayor visibilidad de los procesos de software internos que otras técnicas de análisis de datos. La solución de problemas y la corrección de errores también son más fáciles porque los cálculos son menos complejos.

1.3. ¿Cuáles son las aplicaciones de la regresión logística?

La regresión logística tiene varias aplicaciones del mundo real en muchos sectores diferentes.

- **Fabricación**

Las empresas de fabricación utilizan el análisis de regresión logística para estimar la probabilidad de fallo de las piezas en la maquinaria. Luego, planifican los programas de mantenimiento en función de esta estimación para minimizar los fallos futuros.

- **Sanidad**

Los investigadores médicos planifican la atención y el tratamiento preventivos mediante la predicción de la probabilidad de enfermedad en los pacientes. Utilizan modelos de regresión logística para comparar el impacto de los antecedentes familiares o los genes en las enfermedades.

- **Finanzas**

Las empresas financieras tienen que analizar las transacciones financieras en busca de fraudes y evaluar las solicitudes de préstamos y seguros en busca de riesgos. Estos problemas son adecuados para un modelo de regresión logística porque tienen resultados discretos, como alto riesgo o bajo riesgo y fraudulento o no fraudulento.

- **Marketing**

Las herramientas de publicidad en línea utilizan el modelo de regresión logística para predecir si los usuarios harán clic en un anuncio. Como resultado, los especialistas en marketing pueden analizar las respuestas de los usuarios a diferentes palabras e imágenes y crear anuncios de alto rendimiento con los que los clientes interactuarán.

1.4. ¿Cómo funciona el análisis de regresión?

La regresión logística es una de las diferentes técnicas de análisis de regresión que los científicos de datos utilizan habitualmente en machine learning (ML). Para entender la regresión logística, primero debemos entender el análisis de regresión básica. A continuación, utilizamos un ejemplo de análisis de regresión lineal para demostrar cómo funciona el análisis de regresión.

Identificar la pregunta



Cualquier análisis de datos comienza con una pregunta empresarial. Para la regresión logística, hay que formular la pregunta para obtener resultados concretos:

- ¿Los días de lluvia afectan nuestras ventas mensuales? (sí o no)
- ¿Qué tipo de actividad de tarjeta de crédito realiza el cliente? (autorizado, fraudulento o potencialmente fraudulento)

Recopilar datos históricos

Una vez identificada la pregunta, debe identificar los factores de los datos que intervienen. A continuación, recopilará datos anteriores para todos los factores. Por ejemplo, para responder a la primera pregunta que se muestra arriba, puede recopilar el número de días de lluvia y los datos de ventas mensuales de cada mes en los últimos tres años.

Entrenar el modelo de análisis de regresión

Deberá procesar los datos históricos mediante un software de regresión. El software procesará los diferentes puntos de datos y los conectará matemáticamente mediante ecuaciones. Por ejemplo, si el número de días lluviosos durante tres meses es 3, 5 y 8 y el número de ventas en esos meses es 8, 12 y 18, el algoritmo de regresión conectará los factores con la ecuación:

$$\text{Número de ventas} = 2 * (\text{número de días lluviosos}) + 2$$

Realizar predicciones para valores desconocidos

Para valores desconocidos, el software utiliza la ecuación para hacer una predicción. Si sabe que lloverá durante seis días en julio, el software calculará el valor de venta de julio en 14.

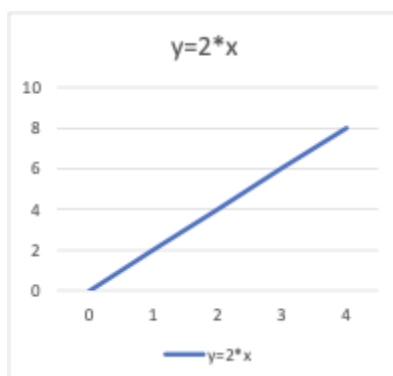
1.5. ¿Cómo funciona el modelo de regresión logística?

Para entender el modelo de regresión logística, primero vamos a entender las ecuaciones y las variables.

- **Ecuaciones**

En matemáticas, las ecuaciones dan la relación entre dos variables: x e y . Puede usar estas ecuaciones, o funciones, para trazar un gráfico a lo largo de los ejes x e y poniendo diferentes valores de x e y . Por ejemplo, si traza el gráfico para la función $y = 2 \cdot x$, obtendrá una línea recta como se muestra a continuación. Por lo tanto, esta función también se denomina función lineal.





- Variables

En estadística, las variables son los factores o atributos de datos cuyos valores varían. Para cualquier análisis, ciertas **variables son independientes o explicativas**. Estos atributos son la causa de un resultado. Otras variables son **variables dependientes o de respuesta**; sus valores dependen de las variables independientes. En general, la regresión logística explora cómo las variables independientes afectan a una variable dependiente al observar los valores de datos históricos de ambas variables.

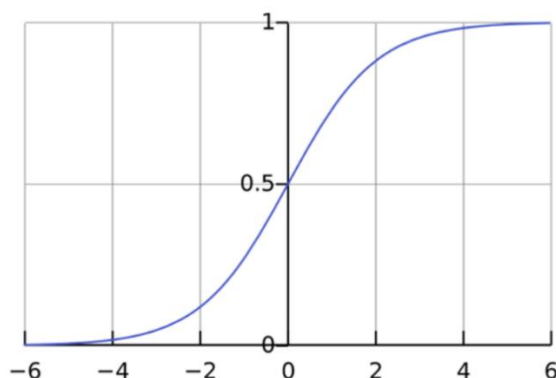
En nuestro ejemplo anterior, x se denomina variable independiente, variable predictora o variable explicativa porque tiene un valor conocido. Y se denomina variable dependiente, variable de resultado o variable de respuesta porque se desconoce su valor.

- Función de regresión logística

La regresión logística es un modelo estadístico que utiliza la función logística, o función **logit**, en matemáticas como la ecuación entre x e y .

$$f(x) = \frac{1}{1 + e^{-x}}$$

Si representamos esta ecuación de regresión logística, obtendremos una curva en S como la que se muestra a continuación.



Como se puede ver, la función **logit** devuelve solo valores entre 0 y 1 para la variable dependiente, al margen de los valores de la variable independiente. Así es como la regresión logística estima el valor de la variable dependiente. Los métodos de regresión logística también modelan ecuaciones entre múltiples variables independientes y una variable dependiente.

- **Análisis de regresión logística con múltiples variables independientes**

En muchos casos, múltiples variables explicativas afectan al valor de la variable dependiente. Para modelar dichos conjuntos de datos de entrada, las fórmulas de regresión logística asumen una relación lineal entre las diferentes variables independientes. Se puede modificar la función sigmoidea y calcular la variable de salida final como:

$$y = f(\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n)$$

$$f(x) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n)}}$$

El símbolo β representa el coeficiente de regresión. El modelo **logit** puede calcular de forma inversa estos valores de coeficientes cuando se le proporciona un conjunto de datos experimentales suficientemente grande con valores conocidos de variables dependientes e independientes.

- **Registrar probabilidades**

El modelo **logit** también puede determinar la relación entre el éxito y el fracaso o registrar las probabilidades.

Por ejemplo, si estaba jugando al póker con sus amigos y ganó cuatro partidos de 10, sus probabilidades de ganar son cuatro sextos, o cuatro de seis, que es la relación entre su éxito y su fracaso. La probabilidad de ganar, por otro lado, es de cuatro sobre 10.

Matemáticamente, sus probabilidades en términos de probabilidad son: $\frac{p}{1-p}$, y sus registros de probabilidades son $(\frac{p}{1-p})$. Se puede representar la función logística como registro de probabilidades como se muestra a continuación:

$$\text{logit función} = \log\left(\frac{p}{1-p}\right)$$

1.6. ¿Cuáles son los tipos de análisis de regresión logística?

Hay tres enfoques para el análisis de regresión logística basados en los resultados de la variable dependiente.

- **Regresión logística binaria**

La regresión logística binaria funciona bien para problemas de clasificación binaria que solo tienen dos resultados posibles. La variable dependiente solo puede tener dos valores, como sí y no o 0 y 1.



Aunque la función logística calcula un rango de valores entre 0 y 1, el modelo de regresión binaria redondea la respuesta a los valores más cercanos. Por lo general, las respuestas por debajo de 0,5 se redondean a 0 y las respuestas por encima de 0,5 se redondean a 1, de modo que la función logística devuelve un resultado binario.

- **Regresión logística multinomial**

La regresión multinomial puede analizar problemas que tienen varios resultados posibles, siempre y cuando el número de resultados sea finito. Por ejemplo, puede predecir si los precios de la vivienda aumentarán un 25 %, 50 %, 75 % o 100 % en función de los datos de población, pero no puede predecir el valor exacto de una casa.

La regresión logística multinomial funciona mapeando los valores de resultado con diferentes valores entre 0 y 1. Como la función logística puede devolver un rango de datos continuos, como 0,1, 0,11, 0,12, etc., la regresión multinomial también agrupa el resultado a los valores más cercanos posibles.

- **Regresión logística ordinal**

La regresión logística ordinal, o el modelo logit ordenado, es un tipo especial de regresión multinomial para problemas en los que los números representan rangos en lugar de valores reales. Por ejemplo, puede utilizar la regresión ordinal para predecir la respuesta a una pregunta de una encuesta en la que se pide a los clientes que clasifiquen su servicio como malo, regular, bueno o excelente en función de un valor numérico, como el número de artículos que le han comprado a lo largo del año.

1.7. ¿Cómo se compara la regresión logística con otras técnicas de ML?

Las dos técnicas comunes de análisis de datos son el análisis de regresión lineal y el aprendizaje profundo.

- **Análisis de regresión lineal**

Como se explicó anteriormente, la regresión lineal modela la relación entre variables dependientes e independientes mediante el uso de una combinación lineal. La ecuación de regresión lineal es

$y = \beta_0 X_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_n X_n + \varepsilon$, donde β_1 a β_n y ε son coeficientes de regresión.

- **Regresión logística vs. regresión lineal**

La regresión lineal predice una variable dependiente continua mediante el uso de un conjunto dado de variables independientes. Una variable continua puede tener un rango de valores, como el precio o la antigüedad. Por lo tanto, la regresión lineal puede predecir los valores reales de la variable dependiente. Puede responder a preguntas como “¿Cuál será el precio del arroz después de 10 años?”

A diferencia de la regresión lineal, la regresión logística es un algoritmo de clasificación. No puede predecir los valores reales de los datos continuos. Puede responder a preguntas como “¿Aumentará el precio del arroz un 50 % en 10 años?”



- **Aprendizaje profundo**

El [aprendizaje profundo](#) utiliza redes neuronales o componentes de software que simulan el cerebro humano para analizar la información. Los cálculos de aprendizaje profundo se basan en el concepto matemático de vectores.

- **Regresión logística frente a aprendizaje profundo**

La regresión logística es menos compleja y requiere menos recursos informáticos que el aprendizaje profundo. Y lo que es más importante, los desarrolladores no pueden investigar ni modificar los cálculos de aprendizaje profundo, debido a su naturaleza compleja y dirigida por la máquina. Por otro lado, los cálculos de regresión logística son transparentes y más fáciles de solucionar.

1.8. Regresión Logística con Python

1.8.1. Ejemplo1 en Python: Predicción de compra en tienda online

Supongamos que queremos predecir si un visitante de una tienda online realizará una compra (Sí/No) basándonos en dos características: el tiempo que pasa en el sitio (en minutos) y la cantidad de productos que añade al carrito. Para ello, generaremos un conjunto de datos sintético con estas características y la variable objetivo "compra" que indica si el visitante compró o no.

```
import numpy as np
import pandas as pd

# Semilla para reproducibilidad
np.random.seed(42)

# Número de muestras
n_samples = 5000

# Generar tiempo en sitio (minutos) entre 1 y 10
tiempo = np.random.uniform(1, 10, n_samples)

# Generar cantidad de productos en carrito entre 0 y 5
productos = np.random.randint(0, 6, n_samples)

# Regla para determinar compra:
# Si tiempo > 5 y productos > 2 → compra = 1, si no → 0
compra = ((tiempo > 5) & (productos > 2)).astype(int)

# Crear DataFrame
data = pd.DataFrame({
    'tiempo': tiempo,
    'productos': productos,
    'compra': compra
})

print(data.head())
print("\nTamaño del dataset:", data.shape)
```



1. Preparación del entorno en Python

```
import pandas as pd
import numpy as np

# Librerías para crear el modelo de regresión logística
from sklearn import linear_model, model_selection
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score

# Librerías para visualización
import matplotlib.pyplot as plt
import seaborn as sb

# Configuración para mostrar gráficos dentro del notebook
%matplotlib inline

# Comentario:
# Este bloque prepara todas las herramientas necesarias para:
# - Cargar y manipular datos (pandas, numpy)
# - Entrenar un modelo de regresión logística (sklearn)
# - Evaluar el modelo (accuracy, matriz de confusión, reporte de clasificación)
# - Visualizar datos y resultados (matplotlib, seaborn)
```

2. Carga del dataset generado

Antes de entrenar el modelo, necesitamos cargar el dataset que generado previamente.

```
# Si ya tienes el DataFrame 'data' generado en memoria, no necesitas hacer nada.
# Si lo guardaste en un archivo CSV, descomenta la siguiente línea:
# data = pd.read_csv("dataset_tienda_online.csv")

# Mostrar primeras filas
data.head()
```

3. División Train/Test

Dividimos los datos en entrenamiento (80%) y prueba (20%). Esto permite evaluar el modelo con datos que nunca ha visto.

```
# DIVISIÓN TRAIN/TEST
X = data[['tiempo', 'productos']] # Variables predictoras
y = data['compra']               # Variable objetivo

# División 80% entrenamiento, 20% prueba
X_train, X_test, y_train, y_test = model_selection.train_test_split(
    X, y, test_size=0.2, random_state=42
)
X_train.shape, X_test.shape
```



4. Entrenamiento del modelo de regresión logística

Creamos el modelo, lo entrenamos y mostramos los coeficientes aprendidos.

```
# ENTRENAMIENTO DEL MODELO

modelo = linear_model.LogisticRegression()
modelo.fit(X_train, y_train)

print("Coeficientes del modelo:", modelo.coef_)
print("Intercept:", modelo.intercept_)
```

5. Evaluación completa del modelo

Predicciones

```
# Predicciones
y_pred = modelo.predict(X_test)

# Métricas
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nMatriz de confusión:\n", confusion_matrix(y_test, y_pred))
print("\nReporte de clasificación:\n", classification_report(y_test, y_pred))
```

Visualización de la frontera de decisión

Esto permite ver cómo el modelo separa las clases en el plano (tiempo vs productos). Es muy útil para interpretar modelos simples.

```
# VISUALIZACIÓN DE LA FRONTERA DE DECISIÓN

# Crear una malla de puntos
x_min, x_max = X['tiempo'].min() - 1, X['tiempo'].max() + 1
y_min, y_max = X['productos'].min() - 1, X['productos'].max() + 1

xx, yy = np.meshgrid(
    np.linspace(x_min, x_max, 300),
    np.linspace(y_min, y_max, 300)
)

# Crear DataFrame con nombres de columnas para evitar el warning
grid = pd.DataFrame({
    'tiempo': xx.ravel(),
    'productos': yy.ravel()
})

# Predicción sobre la malla
Z = modelo.predict(grid)
Z = Z.reshape(xx.shape)
```



```
# Gráfico
plt.figure(figsize=(10, 6))
plt.contourf(xx, yy, Z, alpha=0.3, cmap='coolwarm')

# Puntos reales
sb.scatterplot(data=data, x='tiempo', y='productos', hue='compra', palette='coolwarm')

plt.title("Frontera de decisión - Regresión Logística")
plt.xlabel("Tiempo en sitio (min)")
plt.ylabel("Productos añadidos")
plt.show()
```

1.8.2. Ejemplo2 en Python: Predicción SO (regresión logística multinomial)

Queremos predecir el **Sistema Operativo (SO)** que utiliza un usuario cuando accede a una página web. Las clases posibles son:

- **0** → **Windows**
- **1** → **Macintosh**
- **2** → **Linux**

Para ello, mediremos **cuatro variables numéricas** que un servidor web puede registrar automáticamente y que influyen en el tipo de dispositivo y sistema operativo del usuario:

Variables numéricas utilizadas

1. **ancho_pantalla** (en píxeles)
 - Windows: muy variable
 - macOS: resoluciones altas y estables
 - Linux: resoluciones medias típicas de monitores estándar
2. **alto_pantalla** (en píxeles)
 - Misma lógica que el ancho
3. **velocidad_scroll** (píxeles por segundo)
 - macOS: scroll suave y acelerado
 - Windows: scroll segmentado
 - Linux: scroll más lineal
4. **tiempo_carga_ms** (tiempo de carga de la página en milisegundos)
 - Windows: rendimiento medio
 - macOS: optimización alta → tiempos menores
 - Linux: depende del entorno → tiempos variables

A partir de estas variables, generaremos un dataset sintético de **5000 usuarios**, simulando patrones realistas.

CÓDIGO COMPLETO PARA GENERAR EL DATASET (5000 registros)

Incluye la exportación a CSV con nombre **SO.csv**.

```
import numpy as np
import pandas as pd

# Semilla para reproducibilidad
np.random.seed(42)

# Número de muestras
n = 5000
```



```
# -----
# 1. Generación de variables numéricas
# -----

# Ancho de pantalla (px)
# Windows: muy variado, macOS: alto, Linux: medio
ancho_windows = np.random.normal(1366, 200, int(n*0.6))
ancho_mac = np.random.normal(2560, 150, int(n*0.25))
ancho_linux = np.random.normal(1920, 180, int(n*0.15))

ancho_pantalla = np.concatenate([ancho_windows, ancho_mac, ancho_linux])

# Alto de pantalla (px)
alto_windows = np.random.normal(768, 120, int(n*0.6))
alto_mac = np.random.normal(1600, 100, int(n*0.25))
alto_linux = np.random.normal(1080, 130, int(n*0.15))

alto_pantalla = np.concatenate([alto_windows, alto_mac, alto_linux])

# Velocidad de scroll (px/s)
scroll_windows = np.random.normal(800, 150, int(n*0.6))
scroll_mac = np.random.normal(1200, 200, int(n*0.25))
scroll_linux = np.random.normal(900, 180, int(n*0.15))

velocidad_scroll = np.concatenate([scroll_windows, scroll_mac, scroll_linux])

# Tiempo de carga (ms)
carga_windows = np.random.normal(900, 150, int(n*0.6))
carga_mac = np.random.normal(700, 120, int(n*0.25))
carga_linux = np.random.normal(850, 140, int(n*0.15))

tiempo_carga_ms = np.concatenate([carga_windows, carga_mac, carga_linux])

# -----
# 2. Variable objetivo (SO)
# -----
# 0 = Windows, 1 = Macintosh, 2 = Linux
so = np.array(
    [0]*int(n*0.6) +
    [1]*int(n*0.25) +
    [2]*int(n*0.15)
)

# -----
# 3. Crear DataFrame
# -----
data = pd.DataFrame({
    'ancho_pantalla': ancho_pantalla,
    'alto_pantalla': alto_pantalla,
    'velocidad_scroll': velocidad_scroll,
    'tiempo_carga_ms': tiempo_carga_ms,
```



```
'so': so
})

# -----
# 4. Exportar a CSV
# -----
data.to_csv("SO.csv", index=False)

print("Dataset generado con éxito. Tamaño:", data.shape)
print(data.head())
```

Tabla resumen, mostrando los **niveles numéricos utilizados para discriminar cada Sistema Operativo** en el dataset sintético.

Variable numérica	Windows (0)	Macintosh (1)	Linux (2)	Interpretación
Ancho de pantalla (px)	Normal(1366, 200)	Normal(2560, 150)	Normal(1920, 180)	macOS usa pantallas Retina; Windows es muy variable; Linux suele usar monitores estándar.
Alto de pantalla (px)	Normal(768, 120)	Normal(1600, 100)	Normal(1080, 130)	macOS tiene resoluciones altas; Windows y Linux muestran valores más bajos.
Velocidad de scroll (px/s)	Normal(800, 150)	Normal(1200, 200)	Normal(900, 180)	macOS tiene scroll más suave y rápido; Linux intermedio; Windows más segmentado.
Tiempo de carga (ms)	Normal(900, 150)	Normal(700, 120)	Normal(850, 140)	macOS carga más rápido; Windows más lento; Linux depende del entorno.

1. Preparación del entorno

```
# Librerías para manejo de datos
import pandas as pd
import numpy as np

# Librerías para machine learning
from sklearn import linear_model, model_selection
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score

# Visualización
import matplotlib.pyplot as plt
import seaborn as sb
%matplotlib inline
```

2. Carga del dataset SO.csv

```
# CARGA DEL DATASET
data = pd.read_csv("SO.csv")

# Mostrar primeras filas
data.head()
```



3. Exploración inicial del dataset

```
# EXPLORACIÓN INICIAL
print(data.shape)
print(data.describe())
print(data['so'].value_counts())
sb.pairplot(data, hue='so')
```

4. División Train/Test

```
# DIVISIÓN TRAIN/TEST

X = data[['ancho_pantalla', 'alto_pantalla', 'velocidad_scroll', 'tiempo_carga_ms']]
y = data['so']

X_train, X_test, y_train, y_test = model_selection.train_test_split(
    X, y, test_size=0.2, random_state=42
)

X_train.shape, X_test.shape
```

5. Entrenamiento del modelo (Regresión Logística Multinomial)

```
# ESCALADO DE VARIABLES
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# ENTRENAMIENTO DEL MODELO MULTINOMIAL (corregido)
modelo = linear_model.LogisticRegression(
    solver='lbfgs',
    max_iter=1000
)

modelo.fit(X_train_scaled, y_train)

print("Coeficientes del modelo:")
print(modelo.coef_)
print("\nIntercept:")
print(modelo.intercept_)
```

6. Evaluación completa del modelo

```
# EVALUACIÓN COMPLETA DEL MODELO

# 1. Predicciones usando los datos escalados
y_pred = modelo.predict(X_test_scaled)
```



```
# 2. Accuracy
print("Accuracy del modelo:", accuracy_score(y_test, y_pred))

# 3. Matriz de confusión
print("\nMatriz de confusión:")
print(confusion_matrix(y_test, y_pred))

# 4. Reporte de clasificación
print("\nReporte de clasificación:")
print(classification_report(y_test, y_pred))
```

7. Visualización de la frontera de decisión (solo 2 variables)

Para visualizar una frontera, debemos reducir a 2 variables.

Usaremos:

- ancho_pantalla
- velocidad_scroll

```
# 🧠 VISUALIZACIÓN DE LA FRONTERA DE DECISIÓN (2 variables)

from sklearn.preprocessing import StandardScaler

# Seleccionamos solo dos variables para poder graficar
X2 = data[['ancho_pantalla', 'velocidad_scroll']]
y2 = data['so']

# Escalado de estas dos variables
scaler2 = StandardScaler()
X2_scaled = scaler2.fit_transform(X2)

# Entrenamos un modelo SOLO con estas dos variables
modelo2 = linear_model.LogisticRegression(
    solver='lbfgs',
    max_iter=1000
)
modelo2.fit(X2_scaled, y2)

# Crear malla de puntos para graficar la frontera
x_min, x_max = X2['ancho_pantalla'].min() - 50, X2['ancho_pantalla'].max() + 50
y_min, y_max = X2['velocidad_scroll'].min() - 50, X2['velocidad_scroll'].max() + 50

xx, yy = np.meshgrid(
    np.linspace(x_min, x_max, 300),
    np.linspace(y_min, y_max, 300)
)

# Convertimos la malla en DataFrame
grid = pd.DataFrame({
    'ancho_pantalla': xx.ravel(),
    'velocidad_scroll': yy.ravel()
})
```




```

})

# Escalamos la malla igual que los datos reales
grid_scaled = scaler2.transform(grid)

# Predicción sobre la malla
Z = modelo2.predict(grid_scaled)
Z = Z.reshape(xx.shape)

# Gráfico
plt.figure(figsize=(10, 6))
plt.contourf(xx, yy, Z, alpha=0.3, cmap='coolwarm')

# Puntos reales
sb.scatterplot(
    data=data,
    x='ancho_pantalla',
    y='velocidad_scroll',
    hue='so',
    palette='coolwarm'
)

plt.title("Frontera de decisión - Regresión Logística Multinomial")
plt.xlabel("Ancho de pantalla (px)")
plt.ylabel("Velocidad de scroll (px/s)")
plt.show()

```

8. Interpretación del modelo

```

# INTERPRETACIÓN DE COEFICIENTES

coef_df = pd.DataFrame(modelo.coef_, columns=X.columns)
coef_df['Clase (SO)'] = ['Windows (0)', 'Macintosh (1)', 'Linux (2)']
coef_df

```

BIBLIOGRAFÍA

1. Fundamentos del Análisis de Datos con Python: <https://www.quantum.tech/es/learning-path/ai-engineering>

2. Curso introductorio en Google Colab (gratuito y práctico)

Un curso abierto creado por estudiantes del IEEE, ideal para empezar a manipular datos, automatizar tareas y visualizar información.

https://colab.research.google.com/github/IEEESBITBA/Curso-python/blob/master/Curso_Analisis_de_Datos_Clases/Clase_1_Analisis_de_Datos.ipynb#scrollTo=0006-ePnUur



3. Regresión Logística

<https://aws.amazon.com/es/what-is/logistic-regression/>

https://es.wikipedia.org/wiki/Regresi%C3%B3n_log%C3%ADstica

4. Curso de Análisis de Datos Gratis (Python, Excel y R)

Curso certificado que cubre fundamentos del análisis de datos, limpieza, preparación y técnicas estadísticas básicas.

<https://videocursos.co/curso-de-analisis-de-datos-con-python/>

5. Repositorio “Awesome Data Analysis” (GitHub)

Colección de más de 500 recursos gratuitos sobre análisis de datos, Python, estadística y visualización.

<https://github.com/PavelGrigoryevDS/awesome-data-analysis>

